

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

SPDM801: MASTER THESIS IN COMPUTER SCIENCE

Formalising π LL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

1st June 2026



Abstract

This thesis presents a mechanised formalisation of π LL, a process calculus relating linear logic and session-typed communication, in Lean 4. The formalisation covers the static structure of the full calculus, including session types, processes, environments, hyperenvironments, typing judgements, and substitution principles. To represent binding, the development retains explicit free identifiers while encoding bound occurrences by indices: channel binders are handled through a locally nameless representation with co-finite premises, while polymorphic type binders are treated through type-depth indexing, shifting, and substitution. The operational semantics and session-fidelity development are mechanised for the multiplicative fragment, π MLL, using a derivation-level transition system together with projected process-level and environment-level relations. The completed metatheory establishes the projection of typed transitions to process and resource transitions and develops the reverse lifting argument for all multiplicative cases except the restriction case. The remaining *res* case is isolated to the reconstruction of endpoint-bearing hyperenvironment components after a transition under restriction. The development demonstrates that mechanising the paper theory in this way can relocate implicit reasoning about bound names, freshness, and unordered linear resources into explicit structural invariants. It thereby contributes a mechanised account of π LL and identifies the remaining infrastructure required to complete session fidelity for π MLL.

Resumé

Denne afhandling præsenterer en mekaniseret formalisering af π LL, en proceskalkule, der forbinder lineær logik med sessionstypet kommunikation, i Lean 4. Formaliseringen dækker den statiske struktur af den fulde kalkule, herunder sessionstyper, processer, miljøer, hypermiljøer, *typing judgements* og substitutionsprincipper. Til repræsentation af binding fastholder udviklingen eksplicitte frie navne, mens bundne forekomster repræsenteres ved indeks: kanalbindere håndteres gennem en *locally nameless* repræsentation med co-finite-præmisser, mens polymorfe typebindere behandles gennem typedybdeindeksering, forskydning og substitution. Den operationelle semantik og udviklingen af *session fidelity* er mekaniseret for det multiplikative fragment, π MLL, ved hjælp af et transitionssystem over *typing derivations* samt projicerede transitionsrelationer for processer og hypermiljøer. Den gennemførte metateori etablerer projektionen af typede transitioner inkluderende til proces- og ressourcetransitioner og udvikler det omvendte løfteargument for alle multiplikative tilfælde undtagen restriktionstilfældet. Det resterende *res*-tilfælde er isoleret til rekonstruktionen af de hypermiljøkomponenter, der bærer de restrikerede endepunkter, efter en transition under restriktion. Udviklingen viser, at en mekanisering af papirteorien på denne måde kan flytte implicit ræsonnement om bundne navne, friskhed og uordnede lineære ressourcer til eksplicitte strukturelle invarianter. Afhandlingen bidrager dermed med en mekaniseret redegørelse for π LL og identificerer den resterende infrastruktur, der kræves for at fuldføre sessionsfidelitet for π MLL.

CONTENTS

1	Binding Architecture	3
1.1	Design Goals and the Approach	3
1.2	The Locally Nameless Representation	3
1.3	Opening and Closing Binders	5
1.4	Co-finite Quantification	6
References		7

1 Binding Architecture

This chapter develops the locally nameless binding architecture used throughout the formalisation. We begin by stating the design goals that motivate the approach (Section 1.1), then introduce the locally nameless representation and its core operations (Sections 1.2 and 1.3), and conclude with co-finite quantification (Section 1.4), which governs how inductive definitions interact with binders.

1.1 Design Goals and the Approach

This chapter introduces the binding representation used throughout the formalisation. The representation is motivated by three primary design goals:

- (1) **Eliminate α -equivalence overhead.** Binder structures that differ only in the names chosen for bound occurrences should be represented identically, avoiding explicit reasoning about α -equivalence within the proof assistant.
- (2) **Preserve human readability.** Free identifiers should remain explicit in process terms, typing hyperenvironments, and transition labels, so that the mechanised definitions and proof states remain recognisable relative to the paper presentation.
- (3) **Avoid informal meta-level conventions.** The representation should not rely on Barendregt’s convention or unstated freshness assumptions. Capture avoidance should follow from the representation of bound occurrences, while any remaining freshness requirements should appear explicitly in definitions or proof obligations.

Representation alternatives. A direct representation using named bound variables would remain closest to the paper syntax. However, it would require the mechanisation to define and reason explicitly about capture-avoiding renaming and α -equivalence whenever a transition or substitution passes through a binder. At the opposite extreme, a fully nameless representation could encode channel references solely through indices, making equality of binder structures direct but obscuring the free channel names that occur explicitly in typing hyperenvironments and transition labels. Following Charguéraud [2012], the formalisation instead adopts a locally nameless architecture: free identifiers remain explicit, while bound occurrences are represented by indices. Consequently, binder structures that differ only by the choice of bound names coincide structurally, while the observable names used by the operational semantics remain explicit.

Although both channel names and type variables involve binding, they have different roles in the calculus. Channel names participate in dynamic communication and occur in transition labels and typing hyperenvironments; their binders are therefore opened with explicit fresh names. Type variables remain statically scoped within session types and typing derivations; their binders are managed through type-binder depth, shifting, local closure, and substitution. The formalisation consequently applies the same free/bound separation to both syntactic categories, while adapting the associated proof infrastructure to their respective semantic roles.

The remainder of this chapter develops the operations required by this representation. In place of explicit α -equivalence reasoning, the formalisation must establish properties of opening, closing, local closure, and freshness. These obligations are subsequently reflected in the typing relation and in the metatheoretic proofs.

1.2 The Locally Nameless Representation

The locally nameless representation, as presented by Charguéraud [2012], separates variables into two disjoint syntactic categories: free names and bound indices. This combines the readability of named syntax for free variables with a structural representation of binding.

- (1) **Bound variables** are represented using *De Bruijn indices* [de Bruijn 1972], where each index denotes the distance to its corresponding binder. Since the representation is purely positional, α -equivalent terms become structurally identical.
- (2) **Free variables** retain explicit names, preserving the readability of open terms and allowing standard name-based reasoning about environments and transition labels.

By structurally separating these two concepts, the locally nameless approach simplifies standard operations such as capture-avoiding substitution. Since free variables and bound variables inhabit distinct syntactic categories, substitution on free variables is defined to not interact with bound indices. We apply this theory to both the structural and operational layers of πLL .

LN Syntax for Types. In the original presentation of πLL , polymorphic types rely on standard named variables. Under the locally nameless architecture, we expand the syntax of type variables to explicitly distinguish between free and bound occurrences. The grammar for types is updated as follows:

$$\begin{aligned} T &::= \text{free}(x) \mid \text{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\ A, B &::= \dots \mid \exists.A \mid \forall.A \mid X \mid X^\perp & (\text{where } X \in T) \end{aligned}$$

Here, a free type variable carries a unique identifier x , while a bound type variable carries a De Bruijn index k . For example, the polymorphic type $\forall X. \exists Y. (X \wp Y)$ is structurally represented without names using the new syntax as $\forall. \exists. (\text{bound}(1) \wp \text{bound}(0))$, where 0 points to the inner existential binder and 1 points to the outer universal binder.

LN Syntax for Processes. Similarly, the standard process syntax displayed in ?? is updated to reflect the locally nameless separation of channel endpoints. A channel is split into its bound and free counterparts, where bound channel names are represented by indices, while free channels remain explicit identifiers. This split reflects a semantic distinction: free channel names are observable, appearing in transition labels and typing environments, while bound names are purely structural. Renaming a bound variable never changes the behaviour of a process, whereas renaming a free channel can alter which sessions it participates in. The locally nameless representation makes this distinction explicit by construction.

Naively removing the bound names from the process syntax would turn binding operations such as $x[y].P$ and $x(y).P$ into $x[].P$ and $x().P$ making them syntactically indistinguishable from their non-binding unit counterparts. To combat this we annotate anonymous binders using placeholder symbols. We use $\bullet$ for a bound process name, and $\diamond$ for a bound type variable.

$$\begin{aligned} \text{Channel} &::= \text{free}(x) \mid \text{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\ P, Q &::= \dots \mid x[\bullet].P \mid x(\bullet).P \mid \nu(\bullet, \bullet)P \mid x(\diamond).P & (\text{where } x \in \text{Channel}) \end{aligned}$$

While the visual anonymity of these constructors is evident from the grammar, their mechanical implementation relies on precise index management. For a multi-name binder like restriction, $\nu(\bullet, \bullet)P$, the two dual endpoints are simultaneously mapped to distinct indices within the continuation of P , namely 0 and 1. For communication prefixes like $x(\bullet).P$, the transmitted name is implicitly bound as index 0 within P , while the explicit free channel x is deliberately preserved as the subject. By maintaining explicit free names strictly for these outward-facing communications, the transition labels and operational semantics of the calculus remain completely unaffected by the architectural shift.

While the locally nameless architecture elegantly resolves α -equivalence, moving between abstracted and instantiated forms requires specialised operations. To execute a substitution or evaluate a process across a binder, we must formally define how to open and close these terms.

1.3 Opening and Closing Binders

Because the locally nameless representation separates free and bound variables, standard substitution cannot be applied uniformly under binders. Instead, moving between bound and free representations is handled by two dual operations, namely *opening* and *closing* [Charguéraud 2012].

Opening (Instantiation). Opening is the operation of traversing into a binding construct and replacing the exposed bound index with a concrete free name. Formally, opening a process P at depth k with a free name x is denoted $\{k \mapsto x\}P$.

The operation proceeds via structural recursion. A depth counter, initially set to 0, tracks the number of binders traversed. As the recursion descends through the syntax tree, the counter increments when passing a new binder. Consider being at depth k , then traversing a single-name binder like $x(\bullet).P$ increments the depth to $k + 1$, while traversing a double-name binder like $\nu(\bullet, \bullet)P$ would increment it to $k + 2$.

When the recursion reaches a bound variable $\text{bound}(n)$, the index n is compared against the current depth k . If $n = k$, the variable refers to the binder currently being opened and is replaced by the free name $\text{free}(x)$. Otherwise, the index refers to a different binder and is left unchanged.

Crucially, when the opening operation encounters an existing free variable $\text{free}(y)$, it simply ignores it. This guarantees that opening is entirely capture-avoiding by construction. In the operational semantics, when a single binder like an input prefix $z(\bullet).P$ receives a channel x , the continuation P is opened as $\{0 \mapsto x\}P$. For a restriction $\nu(\bullet, \bullet)P$, the dual endpoints x and y are opened sequentially as $\{0 \mapsto x\}(\{1 \mapsto y\}P)$.

Closing (Abstraction). Closing is the companion abstraction operation to opening. It is the operation of taking an open term and abstracting a specific free name x into a bound index at depth k , denoted $\{k \leftarrow x\}P$. While recursing, bound indices are ignored as they cannot contain free names. When the recursion encounters a free term like $\text{free}(y)$, it checked if $y = x$. If they match, the free name is replaced by $\text{bound}(k)$.

This operation is primarily used when constructing new binders, such as moving a free channel into a restricted scope. To close multiple free names simultaneously, such as abstracting x and y into a restriction, the operations are applied sequentially, by first closing y at depth $k + 1$, followed by closing x at depth k .

Locally Closed Terms. The separation of names and indices introduces a new syntactic anomaly, namely that a term can contain a *dangling* index that does not point to any enclosing binder. For example, the process term $\text{bound}(0)[]$ is syntactically valid, matching the syntax for an empty send, but mathematically it is meaningless because the index 0 has no corresponding binder.

To filter out these meaningless terms, the locally nameless architecture introduces the predicate of being *locally closed*, denoted $\text{lc}(P)$. A term is locally closed if and only if all of its De Bruijn indices are properly bound by an enclosing constructor.

The locally closed predicate excludes terms containing dangling indices and therefore characterises the well-formed syntax of the calculus. Establishing this predicate is not only a static validation step, but rather it forms a key invariant throughout the mechanisation, which guarantees the soundness of the calculus. Generally, any subsequent definition, such as variable substitution, structural congruence, or operational semantics, must be mathematically proven to *preserve* the locally closed property. In practice, this is achieved through the *regularity of typing*, which requires proving that all valid typing derivations inherently produce locally closed terms. Consequently, operations proven to preserve typing will naturally carry this invariant as a structural guarantee.

1.4 Co-finite Quantification

With the syntax partitioned into names and indices, and the mechanics of opening and closing established, the remaining design question is how binder-sensitive inductive relations, in particular typing judgements, should expose fresh representatives.

In paper-based proofs, when a rule requires opening a binder, Barendregt's Variable Convention allows the author simply to state: *pick a fresh name x* . In a proof assistant, this informal choice must instead be represented explicitly in the constructors of the relevant inductive relations. Two standard formulations introduce difficulties for mechanised proofs [Aydemir et al. 2008; Charguéraud 2012]:

- (1) **A single fresh representative.** The rule requires its premise for one chosen name fresh for the relevant syntax and context. While this is sufficient for constructing an individual derivation, the resulting induction hypothesis applies only to the representative selected in that derivation.
- (2) **Unrestricted universal quantification.** The rule requires its premise for every name. This provides a strong induction hypothesis, but is too restrictive for proof construction, since the premise cannot generally hold for names already occurring free in the surrounding objects.

Following Aydemir et al. [2008]; Charguéraud [2012], the formalisation uses co-finite quantification for typing rules that open process binders with fresh channel names. Rather than fixing one particular fresh representative, a constructor records a finite exclusion set L and requires its premise for every name outside that set. Schematically, a rule introducing one fresh channel uses a premise of the form $\forall x \notin L$, while a rule introducing two fresh channels additionally requires the selected names to be distinct.

Choice of freshness discipline. Co-finite quantification occupies the middle ground between the two formulations above. It introduces explicit finite exclusion sets into binder-sensitive constructors, but allows a proof to enlarge the exclusion set to contain every name relevant to the current proof obligation and then select a representative fresh for all of them simultaneously. In the present formalisation, this is used for process binders whose continuations are opened with concrete free channel names. For example, when a proof requires an opened continuation to avoid names occurring in a process, a hyperenvironment, and a transition label, these name sets can all be included in L .

This choice becomes significant in the metatheory. The typing rules provide co-finite premises, whereas ProcStep_m exposes concrete names selected by a process transition. Auxiliary *all fresh* lemmas bridge this mismatch by instantiating a co-finite typing premise with the concrete representative occurring in the transition.

Application to πLL types. Although type variables are represented using the same separation between free and bound occurrences, their metatheory is handled using depth-indexed reasoning rather than co-finite quantification. Type variables remain statically scoped within types and typing derivations rather than being communicated as observable channel names, so introducing finite freshness sets for them would add proof infrastructure not otherwise required by their role in the calculus.

Instead, polymorphic typing rules are formulated using a depth-indexed approach based on a De Bruijn shifting operator. We define the shifting of a type A at cutoff depth d by distance c , denoted mathematically as $\uparrow_d^c A$. Using this infrastructure, consider the typing rule for universal type input, $\forall.B$:

$$\frac{n + 1 \vdash P :: \uparrow_0^1 \Gamma, x : B}{n \vdash x(\diamond).P :: \Gamma, x : \forall.B} \forall_{\text{DB}}$$

Rather than opening the binder with a fresh explicit type variable, the typing judgement increments the depth counter to $n + 1$, bringing index 0 into scope. The surrounding typing environment is shifted by distance 1 at cutoff 0, written $\uparrow_0^1 \Gamma$, so that type-variable occurrences already present in the environment retain their intended binding structure beneath the newly introduced binder.

Application to π LL channels. Channel binders are treated differently because they occur in the dynamic process syntax and are opened with concrete free channel names. For example, consider the typing rule introducing the input prefix $x(\bullet).P$. In the paper presentation, the premise implicitly relies on a fresh session name y . In the mechanised presentation, this freshness requirement is expressed by co-finite quantification together with the opening operation:

$$\frac{\forall y \notin L, \quad n \vdash \{0 \mapsto y\}P :: \Gamma, x : B, y : A}{n \vdash x(\bullet).P :: \Gamma, x : A \wp B} \wp_{\text{LN}}$$

Here, the rule states that the process is well typed when, for every fresh channel name y outside a finite exclusion set L , the continuation opened with y is well typed under the environment extended with that name. The formalisation therefore uses a mixed binding strategy: co-finite opening for dynamic channel binders and depth-indexed reasoning for static type-variable binders.

This mixed strategy determines where the remaining proof overhead appears. Depth-indexed treatment of polymorphic types avoids managing fresh type-variable sets, but requires explicit shifting and weakening lemmas when typing derivations cross polymorphic binders. Co-finite quantification for process channels avoids explicit α -equivalence reasoning and provides strong induction hypotheses, but later requires bridge lemmas when an operational transition chooses concrete fresh names. These costs are reflected in the mechanised definitions and proofs developed in the following chapters.

With the binding architecture now fixed, the following chapter describes its implementation in Lean 4.

References

- Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. 2008. Engineering Formal Metatheory. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (POPL '08). ACM, New York, NY, USA, 3–15. doi:10.1145/1328438.1328443
- Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2
- N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0